OPTIMAL BINARY SEARCH TREE

Using Dynamic Programming

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
P. Revnth (2520030292)
Akhil Chowdary (2520030507)
S. Sri Sai Pradyumn (2520030595)

Team Number: 4

Section: 1

Under the guidance of

<<Guide Name>>

<<Designation, Department>>



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY/ARTIFICIAL INTELLIGENCE & DATA SCIENCE



DECLARATION

The Mini Project Report entitled "**Optimal Binary Search Tree**" is a record of Bonafide work of **P. Revnth – 2520030292, Akhil Chowdary – 2520030507, S. Sri Sai Pradyumn – 2520030595** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering/Computer Science and Information Technology/ Artifical Intelligence & data Science to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

P. Revnth – 2520030292

Akhil Chowdary – 2520030507

S. Sri Sai Pradyumn – 2520030595

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING / COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Optimal Binary Search Tree**" is a bonafide work done and submitted by **P. Revnth – 2520030292, Akhil Chowdary – 2520030507, S. Sri Sai Pradyumn – 2520030595** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering/ Computer Science and Information Technology/Artificial Intelligence & Data Science, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, <<Guide Name>>, <<Designation>>, <Department>> which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P.Venkateshwar Rao (CSE)/Dr. Khaja Shareef (CSIT)/ Dr. Sreenath Itikela (AI&DS)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology/Department of Artificial Intelligence and Data Science* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

P. Revnth – 2520030292

Akhil Chowdary – 2520030507

S. Sri Sai Pradyumn – 2520030595

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	19
9.	References	20

1.ABSTRACT

The Optimal Binary Search Tree (OBST) project is a software application that builds a binary search tree from a set of distinct keys and their search frequencies so that the average search cost is as low as possible. In an ordinary binary search tree, frequently searched keys may sit deep in the tree, which increases the number of comparisons. A balanced tree does not solve this, because it ignores how often each key is accessed.

The system accepts the keys and frequencies, validates them, and applies an interval Dynamic Programming formulation. For every range of keys it tries each key as the root, combines the already computed optimal costs of the left and right sub-ranges, and keeps the root that gives the minimum cost. The cost and root tables are then used to reconstruct the tree, which is displayed together with its minimum expected search cost. The method runs in $O(n^3)$ time and $O(n^2)$ space.

For a sample of four keys with frequencies 4, 2, 6 and 3, the program produced a minimum weighted cost of 26 (1.73 comparisons per search on average) compared with 31 for a frequency-blind balanced tree. The project demonstrates the practical use of Dynamic Programming in solving an optimization problem.

Team Name: Team 4 (Section 1) – Optimal Binary Search Tree

Team Logo (if any): Not applicable

Team Members:

1. P. Revnth, 2520030292
2. Akhil Chowdary, 2520030507
3. S. Sri Sai Pradyumn, 2520030595

2. INTRODUCTION

A Binary Search Tree (BST) is one of the most widely used data structures for storing sorted data. It supports searching, insertion and deletion by repeatedly comparing a key with the current node and moving left or right. The number of comparisons needed to find a key equals its depth in the tree, so the overall speed of a BST depends heavily on its shape rather than only on the number of keys it stores.

In many real systems the keys are not accessed equally often. Frequently used words in a dictionary, popular identifiers in a compiler symbol table or hot keys in a routing table are searched far more often than the rest. If such keys are placed deep in the tree, the total search cost rises even when the tree is balanced, because a balanced tree such as an AVL or Red-Black tree controls the height but ignores the access frequencies.

The **Optimal Binary Search Tree** problem asks: given n distinct keys and the frequency with which each is searched, how should the tree be shaped so that the total frequency-weighted search cost is minimum? It is a classic Dynamic Programming problem, because the optimal tree for a large range of keys is built from optimal trees for smaller ranges, and storing these results avoids repeated work. This project implements that construction in Java. The program takes the keys and their frequencies, builds the cost and root tables, reconstructs the optimal tree, and displays it together with its expected search cost.

The rest of this report is organised as follows. Section 3 reviews related work, Section 4 states the problem and objectives, Section 5 lists the hardware and software used, Section 6 explains the implementation with the main modules, Section 7 presents and discusses the results, Section 8 gives the conclusion and future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

The following earlier works and learning resources on optimal and frequency-aware search trees were studied before designing this project. The sources are listed in the References section.

1. Knuth (1971) – Optimum Binary Search Trees [1]. Knuth gave the classical dynamic programming treatment of the optimal binary search tree problem. The idea is that every sub-tree of an optimal tree must itself be optimal for its range of keys. He also showed that the optimal root of a range moves monotonically, which reduces the running time from $O(n^3)$ to $O(n^2)$. This project follows the standard cubic formulation and keeps the $O(n^2)$ speed-up as future work.

2. Mehlhorn (1975) – Nearly Optimal Binary Search Trees [2]. Mehlhorn studied heuristics that build trees close to the optimal cost without running the full dynamic programme. These methods choose roots by balancing the total frequency on each side, which is much cheaper to compute for large key sets. The trade-off is that the resulting tree is only near-optimal, whereas this project computes the exact minimum cost.

3. Cormen, Leiserson, Rivest and Stein – Introduction to Algorithms [3]. This textbook presents the optimal binary search tree as a canonical dynamic programming example, together with matrix-chain multiplication. It defines the cost table, the root table and the bottom-up order of filling by increasing interval length. It also covers AVL and Red-Black trees, which guarantee O(log n) height but do not use access frequencies. The table-filling scheme of this project is taken from this formulation.

4. Sleator and Tarjan (1985) – Self-Adjusting Binary Search Trees [4]. Splay trees reorganise themselves after every access so that recently used keys move towards the root. They need no frequency information in advance and are known to perform within a constant factor of the best static tree over a long sequence of searches. However, they do not produce a single globally optimal layout for known frequencies, which is exactly what the present project computes.

5. GeeksforGeeks – Dynamic Programming | Set 24 (Optimal Binary Search Tree) [5]. This widely used tutorial explains the optimal substructure and overlapping sub-problems of the problem, first with a naive recursive solution and then with a bottom-up cost table in several

programming languages. Its example programs mainly report the minimum cost. Our project extends this by also recording the optimal roots, reconstructing the tree, and printing its structure.

6. University lecture notes – Radford, Waterloo and Washington [6], [7], [8]. These course notes derive the recurrence for the optimal tree, prove the optimal substructure by a cut-and-paste argument, and compare the optimal tree with a balanced one to show that a balanced tree can be far from optimal when the frequencies are uneven. The Waterloo notes also discuss bounds on the expected cost and approximate alternatives because the exact tree is expensive to compute. These points shaped the comparison with a balanced tree in the Results section.

7. Educative and Montana State lecture material [9], [10]. These resources define the average search time of a tree as the frequency-weighted number of comparisons and write the recurrence using prefix sums of frequencies, so that the range total can be found in constant time instead of re-adding it every time. The same prefix-sum idea is used in our cost computation.

This project: it follows the standard $O(n^3)$ interval dynamic programming formulation to compute the exact optimal cost, validates that all keys are distinct, reconstructs and displays the tree, and compares its cost with a frequency-blind balanced tree.

4. Problem Statement and Objectives

Problem Statement:

To construct a binary search tree for n distinct keys $k_1 < k_2 < \ldots < k_n$ with known search frequencies $f_1, f_2, \ldots, f_n$ such that the total expected search cost $\Sigma\ (f_i \times depth_i)$, with the root at depth 1, is minimum.

Problem Description:

In a binary search tree the depth of a key decides how many comparisons are needed to find it. When keys are searched with unequal frequencies, an unsuitable arrangement raises the average search cost, so the tree must be shaped using the frequencies and not only the key order.

Objectives:

The project was initiated to show how Dynamic Programming solves a real optimization problem and to give a simple tool that makes the effect of key position on search cost visible. The objectives are:

1. To accept a set of distinct keys and their search frequencies as input, and to validate that keys are distinct and frequencies are positive.
2. To apply interval dynamic programming to compute the minimum-cost binary search tree for every sub-range of keys.
3. To record the optimal root of each interval and use it to reconstruct the complete optimal tree.
4. To display the constructed tree structure and report the minimum expected search cost and the average number of comparisons per search.
5. To compare the optimal tree with a frequency-blind balanced tree on the same keys, to show the benefit of frequency-aware construction.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Standard PC or laptop (no special hardware is required)
- Processor: any modern multi-core CPU (Intel i3 or equivalent, or higher)
- RAM: 4 GB minimum, 8 GB recommended
- Storage: less than 100 MB for the project and its tools

Software Requirements:

- Programming language: Java (JDK 11 or later)
- Algorithm: Dynamic Programming (interval DP); data structure: Binary Search Tree
- IDE or editor: IntelliJ IDEA, Eclipse or Visual Studio Code
- Operating system: Windows, Linux or macOS
- Version control: Git and GitHub
- Optional: Graphviz for graphical tree visualization (planned as future work)

6. IMPLEMENTATION

The application is written in Java as a single file, OptimalBST.java, and is divided into four modules: input validation, the dynamic programming computation, tree reconstruction, and display with verification.

Recurrence used. Let W(i, j) be the sum of the frequencies of keys i to j, and let cost[i][j] be the minimum weighted search cost of a tree built from keys i to j, with the root at depth 1. Then

$$cost[i][j] = W(i, j) + \min \text{ over } r = i \ldots j \text{ of } (cost[i][r-1] + cost[r+1][j]), \quad cost = 0 \text{ for an empty range.}$$

The root r that gives the minimum is stored in root[i][j]. Intervals are processed in increasing order of length, so every sub-problem is solved before it is needed. There are $O(n^2)$ intervals and each tries up to n roots, giving $O(n^3)$ time and $O(n^2)$ space.

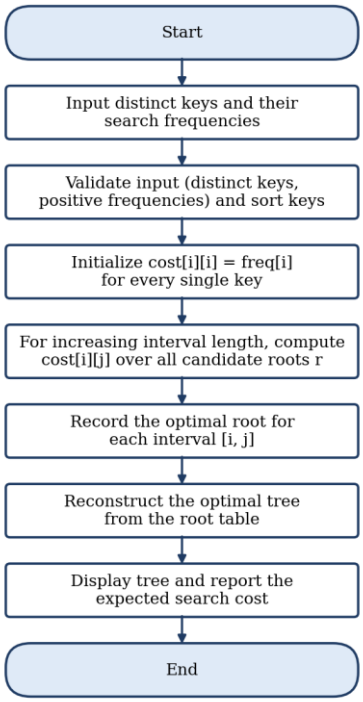


Figure 1: Flow chart of the Optimal BST construction

Module 1 – Node definition and input validation

Defines the tree node and checks that the keys and frequencies are non-empty, of equal length, that no key is repeated and that every frequency is positive, since the formulation assumes distinct keys.

```
static class Node {
    int key, freq;
    Node left, right;
    Node(int key, int freq) { this.key = key; this.freq = freq; }
}


static void validate(int[] keys, int[] freq) {
    if (keys.length == 0 || keys.length != freq.length)
        throw new IllegalArgumentException(
            "Keys and frequencies must be non-empty and of equal length.");
    Set<Integer> seen = new HashSet<>();
    for (int i = 0; i < keys.length; i++) {
        if (!seen.add(keys[i]))
            throw new IllegalArgumentException("Duplicate key found: " + keys[i]);
        if (freq[i] <= 0)
            throw new IllegalArgumentException(
                "Frequency must be positive for key " + keys[i]);
    }
}
```

Module 2 – Interval dynamic programming (cost and root tables)

Builds a prefix-sum array, fills cost[i][i] for single keys, then for each interval length tries every key as root and stores the minimum cost and the chosen root.

```
static int[][] cost, root;


static int computeOptimalCost(int[] freq) {
    int n = freq.length;
    int[] prefix = new int[n + 1];                // prefix[i] = sum of freq[0..i-1]
    for (int i = 0; i < n; i++) prefix[i + 1] = prefix[i] + freq[i];


    cost = new int[n][n];
    root = new int[n][n];
    for (int i = 0; i < n; i++) {                 // base case: single key
        cost[i][i] = freq[i];
        root[i][i] = i;
    }
    for (int len = 2; len <= n; len++) {          // increasing interval length
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            int sum = prefix[j + 1] - prefix[i];  // total frequency of keys i..j
            cost[i][j] = Integer.MAX_VALUE;
            for (int r = i; r <= j; r++) {        // try every key as root
                int left  = (r > i) ? cost[i][r - 1] : 0;
                int right = (r < j) ? cost[r + 1][j] : 0;
                int c = left + right + sum;
                if (c < cost[i][j]) { cost[i][j] = c; root[i][j] = r; }
            }
        }
    }
    return cost[0][n - 1];
}
```

Module 3 – Tree reconstruction from the root table

Starts from the full interval [0, n−1], takes the stored root, and recursively builds the left and right sub-trees from the sub-intervals on either side of it.

```
static Node buildTree(int[] keys, int[] freq, int i, int j) {
    if (i > j) return null;
    int r = root[i][j];
    Node node = new Node(keys[r], freq[r]);
    node.left  = buildTree(keys, freq, i, r - 1);
    node.right = buildTree(keys, freq, r + 1, j);
    return node;
}
```

Module 4 – Display, verification and driver

Prints the tree sideways, recomputes the cost directly from the built tree as a check, builds a balanced tree for comparison, and runs the three test cases.

```
static void printTree(Node node, String prefix, boolean isLeft, boolean isRoot) {
    if (node == null) return;
    if (isRoot) System.out.println("ROOT -> " + node.key + " (freq " + node.freq + ")");
    else        System.out.println(prefix + (isLeft ? "L--- " : "R--- ")
                       + node.key + " (freq " + node.freq + ")");
    String childPrefix = isRoot ? "   " : prefix + "     ";
    printTree(node.left,  childPrefix, true,  false);
    printTree(node.right, childPrefix, false, false);
}


// sum(freq * depth), root at depth 1
static int weightedCost(Node node, int depth) {
    if (node == null) return 0;
    return node.freq * depth + weightedCost(node.left, depth + 1)
                             + weightedCost(node.right, depth + 1);
}


// frequency-blind baseline: middle key of each range becomes the root
static Node balancedTree(int[] keys, int[] freq, int lo, int hi) {
    if (lo > hi) return null;
    int mid = (lo + hi) / 2;
    Node n = new Node(keys[mid], freq[mid]);
    n.left = balancedTree(keys, freq, lo, mid - 1);
    n.right = balancedTree(keys, freq, mid + 1, hi);
    return n;
}


static void run(String title, int[] keys, int[] freq) {
    System.out.println("=== " + title + " ===");
    try { validate(keys, freq); }
    catch (IllegalArgumentException e) {
        System.out.println("Input error: " + e.getMessage() + "\n"); return;
    }


    // sort keys (and carry frequencies along) so that BST order holds
    Integer[] idx = new Integer[keys.length];
    for (int i = 0; i < idx.length; i++) idx[i] = i;
    Arrays.sort(idx, Comparator.comparingInt(a -> keys[a]));
    int[] k = new int[keys.length], f = new int[keys.length];
    for (int i = 0; i < idx.length; i++) { k[i] = keys[idx[i]]; f[i] = freq[idx[i]]; }


    System.out.println("Keys        : " + Arrays.toString(k));
    System.out.println("Frequencies : " + Arrays.toString(f));
    int total = Arrays.stream(f).sum();
    int best = computeOptimalCost(f);
    Node tree = buildTree(k, f, 0, k.length - 1);


    System.out.println("\nOptimal BST structure:");
```

```
    printTree(tree, "", false, true);

    System.out.println("\nMinimum weighted search cost : " + best);
    System.out.println("Sum of frequencies           : " + total);
    System.out.printf ("Average search cost          : %.2f comparisons%n",
                       (double) best / total);
    System.out.println("Cost re-computed from tree   : " + weightedCost(tree, 1));
    int bal = weightedCost(balancedTree(k, f, 0, k.length - 1), 1);
    System.out.printf ("Balanced (frequency-blind)   : %d  (avg %.2f)%n%n",
                       bal, (double) bal / total);
}

public static void main(String[] args) {
    run("Test 1: 4 keys",
        new int[]{10, 20, 30, 40}, new int[]{4, 2, 6, 3});
    run("Test 2: 6 keys (unsorted)",
        new int[]{50, 10, 40, 20, 60, 30}, new int[]{3, 34, 8, 12, 2, 21});
    run("Test 3: duplicate key",
        new int[]{10, 20, 20, 40}, new int[]{4, 2, 6, 3});
}
```

7. RESULTS & DISCUSSION

The program was run on three test cases. Console outputs are shown below, each with a short explanation.

Test 1 – Four keys (sample from the project presentation)

Key	10 (A)	20 (B)	30 (C)	40 (D)
Search frequency	4	2	6	3

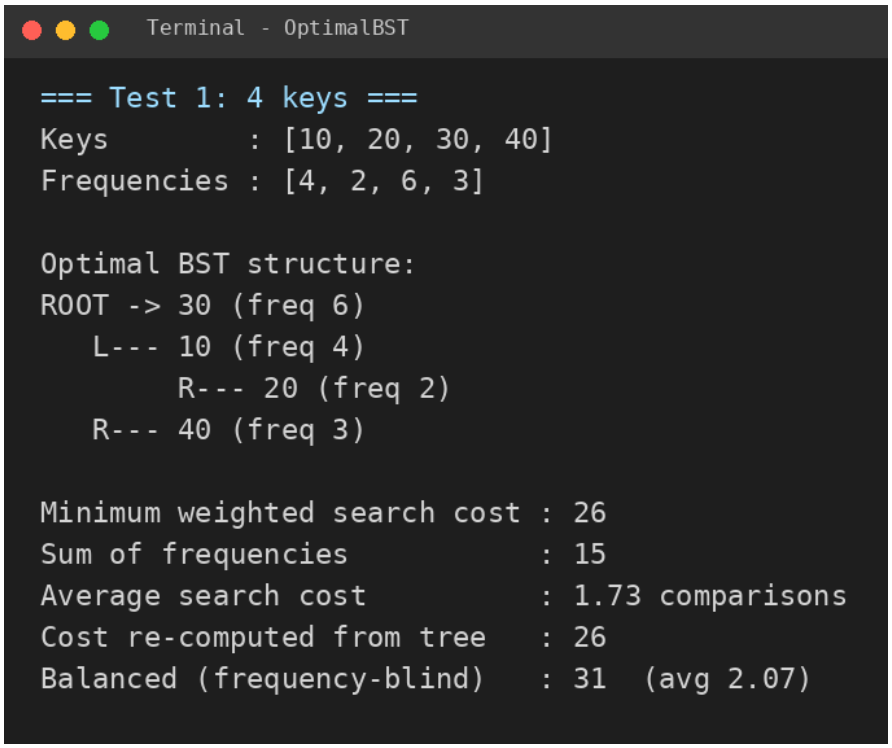


Figure 2: Console output for Test 1

The minimum weighted cost is 26 for a total frequency of 15, which is about 1.73 comparisons per search. The cost recomputed from the built tree also gives 26, confirming the table value.

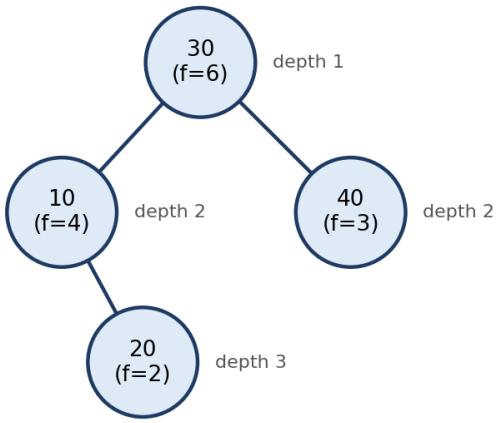


Figure 3: Optimal BST for Test 1

Key 30 (frequency 6) becomes the root; key 10 and key 40 are at depth 2, and key 20 is at depth 3. The cost is 6(1) + 4(2) + 3(2) + 2(3) = 26.

cost[i][j] (optimal root)	**j = 10**	**j = 20**	**j = 30**	**j = 40**
i = 10	4 (10)	8 (10)	20 (30)	26 (30)
i = 20	–	2 (20)	10 (30)	16 (30)
i = 30	–	–	6 (30)	12 (30)
i = 40	–	–	–	3 (40)

Table 1: Dynamic programming table for Test 1

Each cell holds the minimum cost for the range of keys from i to j and, in brackets, the root chosen for that range. The answer is in the top-right cell: 26 with root 30.

Test 2 – Six keys entered in unsorted order

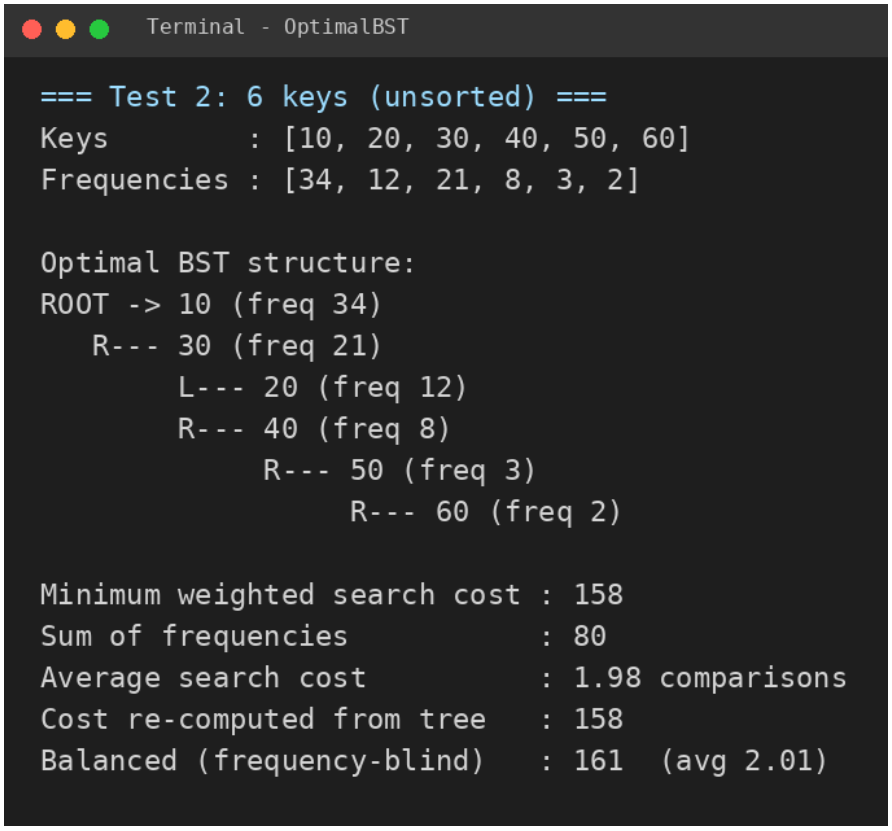


Figure 4: Console output for Test 2

The program sorts the keys first, so the input order does not matter. The heaviest key, 10 (frequency 34), becomes the root even though it is the smallest key, giving a cost of 158 against 161 for the balanced tree.

Test 3 – Input validation (duplicate key)

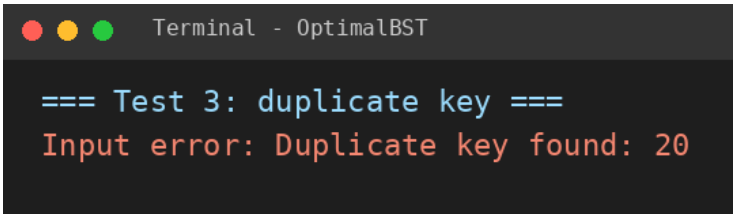


Figure 5: Console output for Test 3

The key 20 is repeated, so the program stops with a clear error message instead of building a wrong tree. This matches the assumption of distinct keys made by the formulation.

Comparison with a frequency-blind balanced tree

Test	Keys (n)	Optimal BST cost	Balanced BST cost	Reduction
Test 1	4	26 (avg 1.73)	31 (avg 2.07)	16.1 %
Test 2	6	158 (avg 1.98)	161 (avg 2.01)	1.9 %

Table 2: Optimal versus balanced tree (balanced tree = middle key of each range chosen as root)

Discussion. The optimal tree is never worse than the balanced one, and the saving depends on how uneven the frequencies are: it is clear in Test 1 (16.1 %) and small in Test 2, where the frequencies are more spread out. The output of Test 1 was also checked by hand against all four possible roots, and the cost recomputed from every reconstructed tree matched the table value. The $O(n^3)$ time and $O(n^2)$ space are acceptable for the small and medium key sets used here, but become costly for very large inputs, which motivates the $O(n^2)$ speed-up listed under future scope.

8. CONCLUSION & FUTURE SCOPE

The project achieved its objectives. A Java program was designed and implemented that accepts distinct keys with their search frequencies, validates them, and uses interval dynamic programming to compute the minimum expected search cost in $O(n^3)$ time by filling the cost and root tables. The optimal tree is reconstructed from the root table and displayed with its cost. On the sample input the cost was 26 (about 1.73 comparisons per search) against 31 for a balanced tree on the same keys, and the results were checked by recomputing the cost from the tree and by hand for the sample. This confirms that building a tree with the access frequencies in mind can reduce the average search cost compared with a frequency-blind balanced tree.

Future Scope

Near term: implement Knuth's $O(n^2)$ optimization, which uses the monotonicity of optimal roots to speed up construction for larger key sets; extend the model with unsuccessful-search (dummy key) frequencies to match the general OBST formulation.

Medium term: add an interactive graphical visualization (for example with Graphviz) to draw the constructed tree; support dynamic updates such as insertions and deletions using ideas from weighted or self-adjusting trees like splay trees.

Long term: apply the technique to real frequency-driven lookup systems such as compiler symbol tables and dictionary or auto-complete engines, and study hybrid approaches that combine a static optimal layout with adaptive restructuring.

9. REFERENCES

[1] D. E. Knuth, "Optimum binary search trees," Acta Informatica, vol. 1, no. 1, pp. 14–25, 1971. doi:10.1007/BF00264289

[2] K. Mehlhorn, "Nearly optimal binary search trees," Acta Informatica, vol. 5, no. 4, pp. 287–295, 1975.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009 (Section 15.5, Optimal binary search trees).

[4] D. D. Sleator and R. E. Tarjan, "Self-adjusting binary search trees," Journal of the ACM, vol. 32, no. 3, pp. 652–686, 1985. doi:10.1145/3828.3835

[5] GeeksforGeeks, "Dynamic Programming | Set 24 (Optimal Binary Search Tree)." http://www.geeksforgeeks.org/dynamic-programming-set-24-optimal-binary-search-tree/

[6] Radford University, "Dynamic Programming – Optimal Binary Search Trees," lecture notes. https://sites.radford.edu/~nokie/classes/360/dp-opt-bst.html

[7] University of Waterloo, CS 466/666 (Fall 2008), "Topic 1: Optimal Binary Search Trees." https://student.cs.uwaterloo.ca/~cs466/Old_courses/F08/Topic1_OptBST.pdf

[8] University of Washington, CSE 417 (Winter 2018), "Dynamic Programming (part 4): Sub-problems on Trees," Lecture 13. https://courses.cs.washington.edu/courses/cse417/18wi/lectures/lec13-dynamic-programming-4.pdf

[9] Educative, "Optimal Binary Search Trees – Dynamic Programming." https://www.educative.io/courses/mastering-algorithms-for-problem-solving-in-java/optimal-binary-search-trees-dynamic-programming

[10] Montana State University, CSCI 223 (Fall 2004), "Optimal Binary Search Trees," lecture. https://www.cs.montana.edu/courses/fall2004/223/Lectures/optimalBST.html